Deep Learning · Day 24 of 30 · about 60 minutes

Gradient descent and a 2-layer network

🎯 Today's goal

By the end of this hour you will:

- measure how wrong a model is with a **loss** (mean squared error)
- understand **gradient descent**: follow the slope downhill, one small step at a time
- see what a **learning rate** that is too small or too big does
- fit a straight line with gradient descent and compare it with Day 17
- build a **2-layer neural network** in NumPy, train it with **backpropagation**, and solve XOR

🤔 Why this matters

Yesterday's perceptron rule only works for a single neuron with a step. Real networks have millions of weights, and they all learn with the same simple idea: **gradient descent**.

Imagine you are on a hillside in thick fog and want to reach the bottom of the valley. You can't see the valley, but you can feel which way the ground slopes under your feet. So you take a small step downhill, feel again, take another step… and eventually you arrive. In machine learning, the "height" is how wrong the model is, and the "position" is the values of its weights. Many of the models you trained with `.fit()` in the last week, and every chatbot, learn this way.

📏 Measuring "how wrong": the loss

Before a model can improve, it needs one number that says how wrong it is. That number is called the **loss**. Smaller is better; 0 means perfect. A very common loss is the **mean squared error** (MSE): take each error (prediction − truth), square it so negatives don't cancel positives and big mistakes count extra, then take the average.

```
import numpy as np

def mse(y_true, y_pred):
    return np.mean((y_pred - y_true) ** 2)

y_true = np.array([3.0, 5.0, 7.0])
good_guess = np.array([2.9, 5.2, 6.8])
bad_guess = np.array([1.0, 8.0, 4.0])

print("errors:", good_guess - y_true)
print(f"MSE of the good guess: {mse(y_true, good_guess):.3f}")
print(f"MSE of the bad guess:  {mse(y_true, bad_guess):.3f}")
```

```
errors: [-0.1  0.2 -0.2]
MSE of the good guess: 0.030
MSE of the bad guess:  7.333
```

⛰️ A problem with one weight

Start tiny. A café recorded how much five customers paid for 1 to 5 cups of coffee (made-up numbers). Our "model" has a single weight `w`, the price per cup: `prediction = w × cups`. Which `w` gives the smallest loss? Let's try a few:

```
cups = np.array([1, 2, 3, 4, 5])
paid = np.array([2.9, 6.1, 9.0, 11.8, 15.2])   # made-up prices

for w in [1, 2, 3, 4, 5]:
    print(f"w = {w}:  loss = {mse(paid, w * cups):.2f}")
```

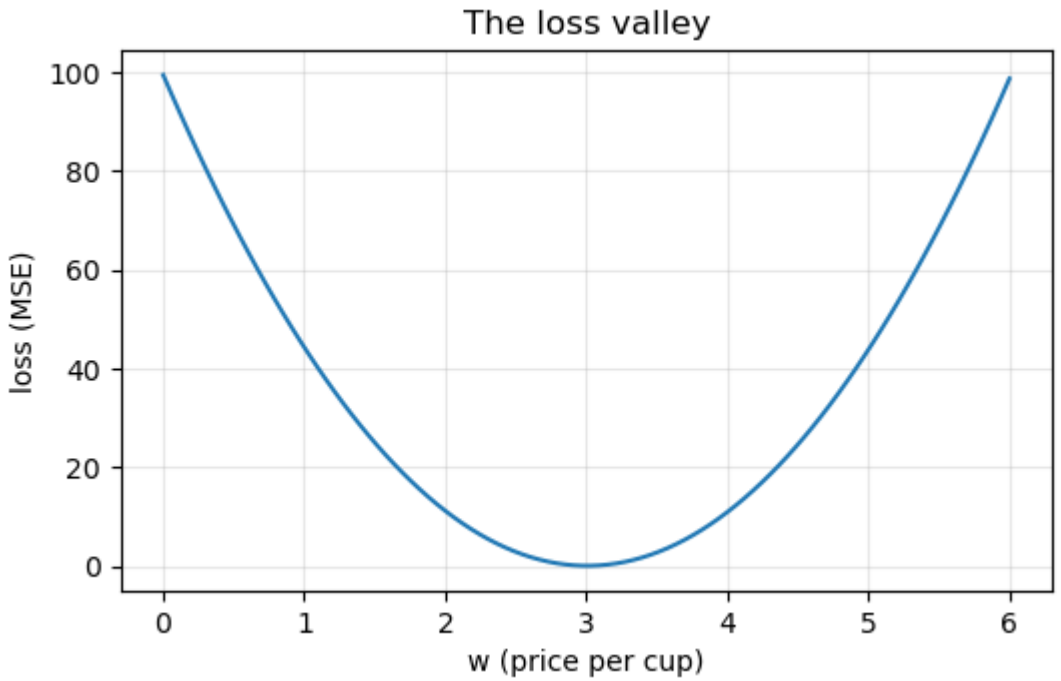
```
w = 1:  loss = 44.26
w = 2:  loss = 11.14
w = 3:  loss = 0.02
w = 4:  loss = 10.90
w = 5:  loss = 43.78
```

The loss for every possible `w` forms a valley. We want its lowest point:

```
import matplotlib.pyplot as plt

w_values = np.linspace(0, 6, 100)
losses = [mse(paid, w * cups) for w in w_values]

plt.figure(figsize=(6, 3.5))
plt.plot(w_values, losses)
plt.xlabel("w (price per cup)")
plt.ylabel("loss (MSE)")
plt.title("The loss valley")
plt.grid(alpha=0.3)
plt.show()
```



With one weight we could simply look at the chart. A real network has millions of weights, so we can never draw this valley. We need a way to walk down it blind, like the hiker in the fog.

📐 The slope tells you which way is downhill

At any `w`, the ground has a **slope**: how much the loss goes up when `w` grows a tiny bit.

- Slope **positive** → the loss rises to the right → step **left** (make `w` smaller).
- Slope **negative** → the loss falls to the right → step **right** (make `w` bigger).
- Slope near **0** → you're at the bottom.

The slope is also called the **gradient** (with many weights, the gradient is the list of slopes, one per weight). You can measure it by nudging `w` a tiny bit and seeing how the loss changes. Calculus

gives a formula that does the same thing exactly; for our MSE it is `mean(2 × error × cups)`. You don't need the calculus, just the idea:

```
def slope(w):
    error = w * cups - paid
    return np.mean(2 * error * cups)     # the formula from calculus

for w in [1.0, 3.0, 5.0]:
    tiny = 0.0001
    nudged = (mse(paid, (w + tiny) * cups) - mse(paid, w * cups)) / tiny
    print(f"w = {w}:  by nudging {nudged:7.2f}   by formula {slope(w):7.2f}")
```

```
w = 1.0:  by nudging  -44.12   by formula  -44.12
w = 3.0:  by nudging   -0.12   by formula   -0.12
w = 5.0:  by nudging   43.88   by formula   43.88
```

Both ways agree. At `w = 1` the slope is negative (go right), at `w = 5` it is positive (go left), and at `w = 3` it is almost 0: we're nearly at the bottom.

🚶 Gradient descent: step downhill, repeat

The whole algorithm is one line, repeated:

```
w = w − learning_rate × slope
```

The minus sign makes us go *against* the slope, which is downhill. The **learning rate** (from yesterday) sets the step size. Let's start from a bad guess, `w = 0`:

```
w = 0.0
lr = 0.01

for step_number in range(1, 16):
    w = w - lr * slope(w)
    print(f"step {step_number:2}:  w = {w:.3f}  loss = {mse(paid, w * cups):.3f}")
```

```
step  1:  w = 0.661  loss = 60.470
step  2:  w = 1.177  loss = 36.798
step  3:  w = 1.579  loss = 22.396
step  4:  w = 1.893  loss = 13.633
step  5:  w = 2.138  loss = 8.302
step  6:  w = 2.329  loss = 5.059
step  7:  w = 2.478  loss = 3.085
step  8:  w = 2.594  loss = 1.885
step  9:  w = 2.684  loss = 1.154
step 10:  w = 2.755  loss = 0.710
step 11:  w = 2.810  loss = 0.440
step 12:  w = 2.853  loss = 0.275
step 13:  w = 2.887  loss = 0.175
step 14:  w = 2.913  loss = 0.114
step 15:  w = 2.933  loss = 0.077
```

`w` walks towards about 3.0 (a cup costs about 3) and the loss shrinks with every step. Notice the steps get smaller near the bottom, because the slope gets flatter.

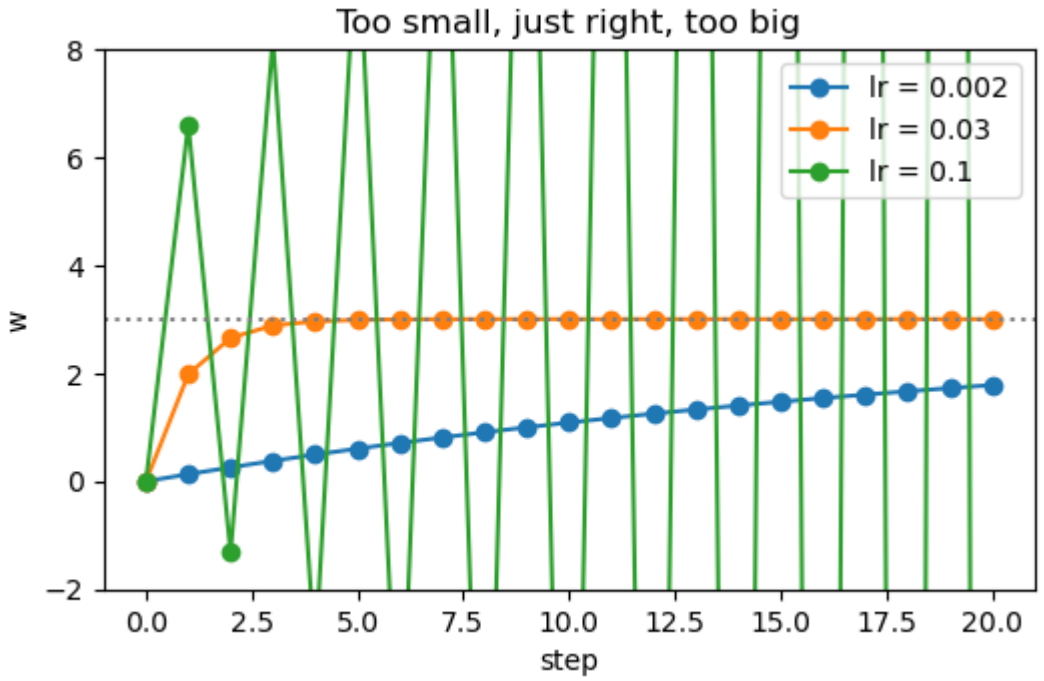
Choosing the learning rate

Too small and you crawl. Too big and you jump right over the valley, landing higher on the other side each time. Let's watch `w` during 20 steps with three learning rates:

```
plt.figure(figsize=(6, 3.5))

for lr in [0.002, 0.03, 0.1]:
    w = 0.0
    history = [w]
    for step_number in range(20):
        w = w - lr * slope(w)
        history.append(w)
    plt.plot(history, marker="o", label=f"lr = {lr}")

plt.axhline(3.0, color="gray", linestyle=":")   # the bottom of the valley
plt.ylim(-2, 8)
plt.xlabel("step")
plt.ylabel("w")
plt.title("Too small, just right, too big")
plt.legend()
plt.show()
```



- **lr = 0.002**: safe but slow, still far from 3 after 20 steps.
- **lr = 0.03**: gets there in a few steps.
- **lr = 0.1**: zigzags further away each step and shoots off the chart. This is called *diverging*.

Picking the learning rate is a matter of trying a few values, and it is one of the most important settings in deep learning.

📈 Fitting a line with gradient descent

On Day 17 you fitted a line `y = a·x + b` with `np.polyfit`. Now let's find `a` and `b` with gradient descent instead. There are two weights now, so each step uses two slopes (the gradient). The data is made up: study hours and test scores for 20 students.

```
rng = np.random.default_rng(42)
hours = np.round(rng.uniform(0, 10, 20), 1)
score = np.round(40 + 5 * hours + rng.normal(0, 4, 20), 1)   # made-up scores

a, b = 0.0, 0.0
lr = 0.02
loss_history = []

for step_number in range(2000):
    error = a * hours + b - score
    loss_history.append(np.mean(error ** 2))
    a = a - lr * np.mean(2 * error * hours)   # slope for a
    b = b - lr * np.mean(2 * error)           # slope for b

print(f"gradient descent: a = {a:.3f}, b = {b:.3f}")
a_fit, b_fit = np.polyfit(hours, score, 1)
print(f"np.polyfit:       a = {a_fit:.3f}, b = {b_fit:.3f}")
```

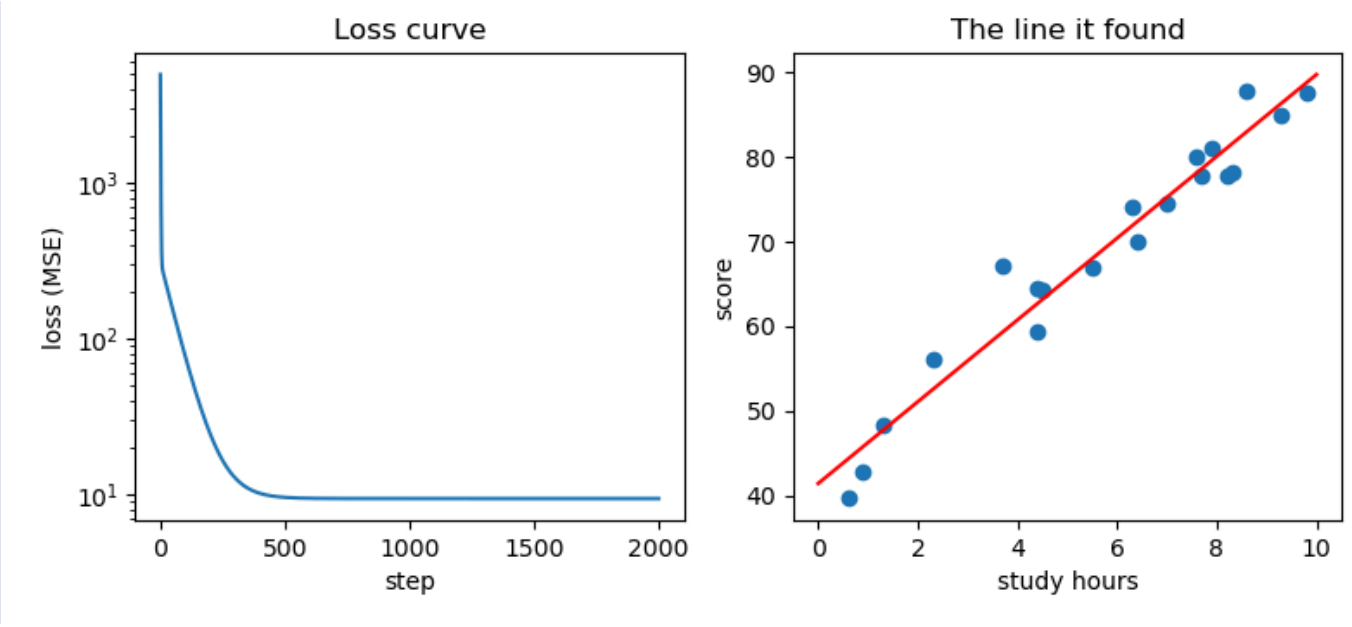
```
gradient descent: a = 4.837, b = 41.382
np.polyfit:       a = 4.837, b = 41.382
```

Same answer! `np.polyfit` uses a ready-made formula that only exists for straight lines. Gradient descent needs no formula, which is exactly why it works for neural networks. Here is the **loss curve** (loss after each step) next to the fitted line:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))

left.plot(loss_history)
left.set_yscale("log")        # each grid step is 10 times bigger
left.set_xlabel("step")
left.set_ylabel("loss (MSE)")
left.set_title("Loss curve")

right.scatter(hours, score)
right.plot([0, 10], [b, a * 10 + b], color="red")
right.set_xlabel("study hours")
right.set_ylabel("score")
right.set_title("The line it found")
plt.show()
```



🧱 A 2-layer network

Yesterday one neuron failed at XOR, because it can only draw one straight line. The fix is to put a **layer** of neurons in between:

- **Input layer**: the 2 inputs (no calculation).
- **Hidden layer**: 4 neurons. Each looks at both inputs and draws its own line. "Hidden" because we never see its answers directly.
- **Output layer**: 1 neuron that combines the 4 hidden answers into the final answer.

Instead of a weight list per neuron, a whole layer's weights go in one 2D array (a **matrix**). `W1` has shape `(2, 4)`: 2 inputs × 4 hidden neurons, one column per neuron. So `X @ W1` computes all 4 neurons for all 4 examples at once. We start with small random weights (seeded, so you get the same numbers), and every neuron uses the sigmoid. Running the network from input to output is called the **forward pass**:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])        # XOR, one answer per row

rng = np.random.default_rng(42)
W1 = rng.normal(size=(2, 4))    # input → hidden: 2 × 4 weights
b1 = np.zeros(4)                # one bias per hidden neuron
W2 = rng.normal(size=(4, 1))    # hidden → output: 4 × 1 weights
b2 = np.zeros(1)

# Forward pass
H = sigmoid(X @ W1 + b1)        # hidden answers, shape (4, 4)
out = sigmoid(H @ W2 + b2)      # final answers, shape (4, 1)

print("shapes:", X.shape, H.shape, out.shape)
# ravel() makes the (4, 1) column one flat row
print("untrained output:", np.round(out.ravel(), 2))
print("loss:", round(mse(y, out), 4))
```

```
shapes: (4, 2) (4, 4) (4, 1)
untrained output: [0.6  0.65 0.72 0.74]
loss: 0.2775
```

Untrained, the network just guesses. Shapes must line up at every layer: the hidden answers `H` go into `W2`, not `W2` into the inputs. Mixing up the layers is a classic slip:

💡 This example has a mistake on purpose. Run it and read the error.

```
print(X.shape, W2.shape)
X @ W2      # oops: W2 expects the 4 hidden answers, not the 2 inputs
```

```
(4, 2) (4, 1)
```

⚠️ ValueError

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 2, in <module>
    X @ W2      # oops: W2 expects the 4 hidden answers, not the 2 inputs
    ~~^~~~
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
gufunc signature (n?,k),(k,m?)->(n?,m?) (size 4 is different from 2)
```

🔙 Backpropagation: passing the blame backwards

To use gradient descent we need the slope of the loss for *every* weight. The trick for getting them all is called **backpropagation** ("backprop"). Think of a group project that got a bad grade: the final presenter (the output neuron) knows how wrong the result was. It passes the blame back to the team members (hidden neurons), each in proportion to how much it listened to them (its weights). Then everyone adjusts a little.

One new tool first. `.T` **transposes** an array: rows become columns. Backprop needs it to send things the opposite way through a layer:

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.shape)
print(A.T)
print(A.T.shape)
```

```
(2, 3)
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
```

Here is one complete training step. Read the comments line by line:

1. **Forward pass:** compute `H` and `out`.
2. **Output blame:** the error `out - y`, times `out × (1 - out)`. That second part is the sigmoid's own slope: a neuron that is already very sure (near 0 or 1) changes little.
3. **Gradient for W2:** blame × the hidden answers that fed in, like yesterday's "error × input".
4. **Hidden blame:** send the output blame back through `W2`, then times each hidden neuron's sigmoid slope.
5. **Gradient for W1:** hidden blame × the inputs.
6. **Update** every weight and bias: `W = W - lr × gradient`.

```
lr = 1.0
print("loss before:", round(mse(y, out), 4))

# 1. Forward pass
H = sigmoid(X @ W1 + b1)
out = sigmoid(H @ W2 + b2)

# 2. Output blame
d_out = (out - y) * out * (1 - out)

# 3. Gradients for the output layer
grad_W2 = H.T @ d_out
grad_b2 = d_out.sum(axis=0)

# 4. Hidden blame: send it back through W2
d_hidden = (d_out @ W2.T) * H * (1 - H)

# 5. Gradients for the hidden layer
grad_W1 = X.T @ d_hidden
grad_b1 = d_hidden.sum(axis=0)

# 6. Update: a small step downhill for every weight
W2 -= lr * grad_W2
b2 -= lr * grad_b2
W1 -= lr * grad_W1
b1 -= lr * grad_b1

out = sigmoid(sigmoid(X @ W1 + b1) @ W2 + b2)
print("loss after: ", round(mse(y, out), 4))
```

```
loss before: 0.2775
loss after:  0.2583
```

Small print: the exact MSE slope also has a factor 2 and a division by the number of examples. We leave them out: they only change the step size, and the learning rate already controls that.

The loss went down a little. Now repeat the same step 5000 times (5000 **epochs**, since each step uses all 4 examples). It takes a second or two:

```
rng = np.random.default_rng(42)          # start again from fresh random weights
W1 = rng.normal(size=(2, 4))
b1 = np.zeros(4)
W2 = rng.normal(size=(4, 1))
b2 = np.zeros(1)
lr = 1.0
xor_losses = []

for epoch in range(5000):
    H = sigmoid(X @ W1 + b1)
    out = sigmoid(H @ W2 + b2)
    xor_losses.append(mse(y, out))

    d_out = (out - y) * out * (1 - out)
    d_hidden = (d_out @ W2.T) * H * (1 - H)

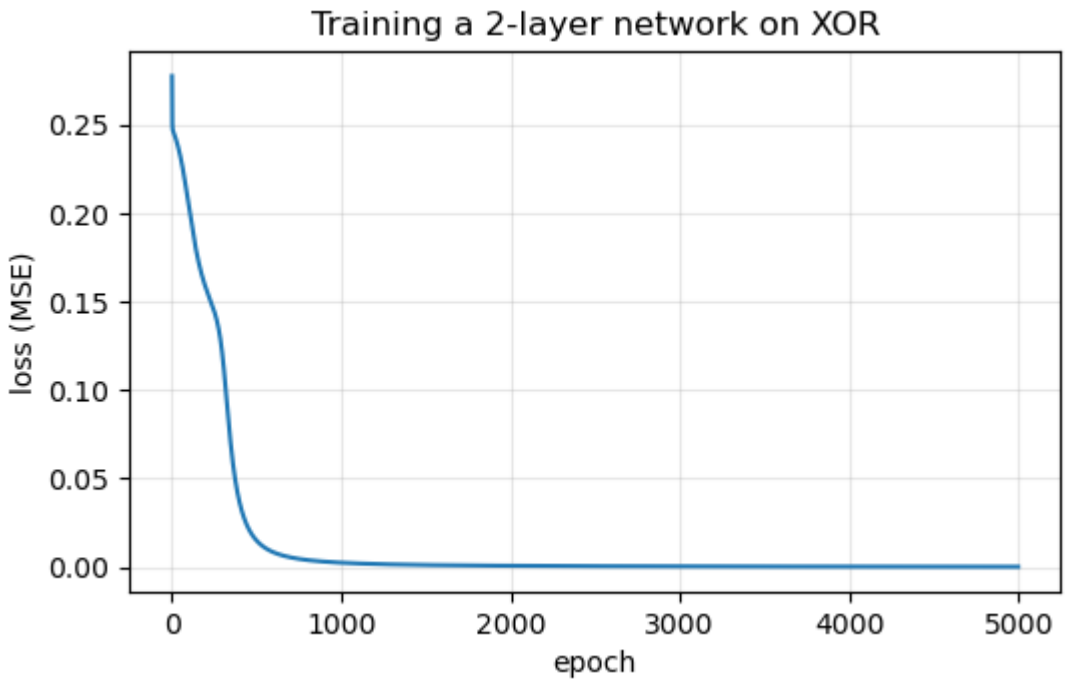
    W2 -= lr * (H.T @ d_out)
    b2 -= lr * d_out.sum(axis=0)
    W1 -= lr * (X.T @ d_hidden)
    b1 -= lr * d_hidden.sum(axis=0)

print("wanted:  ", y.ravel())
print("output:  ", np.round(out.ravel(), 3))
print("rounded: ", np.round(out.ravel()).astype(int))   # 1.0 becomes 1
```

```
wanted:   [0 1 1 0]
output:   [0.009 0.984 0.986 0.019]
rounded:  [0 1 1 0]
```

🎉 XOR solved! The outputs are close to 0, 1, 1, 0. The hidden neurons each learned their own line, and the output neuron learned how to combine them. The loss curve shows the learning:

```
plt.figure(figsize=(6, 3.5))
plt.plot(xor_losses)
plt.xlabel("epoch")
plt.ylabel("loss (MSE)")
plt.title("Training a 2-layer network on XOR")
plt.grid(alpha=0.3)
plt.show()
```



The loss drops fast during the first few hundred epochs, then flattens out near 0: after that the network is only polishing its answers. Loss curves like this are the first thing deep learning engineers look at when they train a model.

⚠️ Common mistakes

- Adding the slope instead of subtracting it: `w + lr * slope` walks *uphill* and the loss grows.
- A learning rate that is too big: the loss jumps around or explodes. Try 10 times smaller.
- A learning rate that is too small: nothing seems to happen. Try 10 times bigger, or more steps.
- Shape mismatches between layers. Print the `.shape` of every array.
- Starting all weights at exactly 0 in a network: every hidden neuron then learns the same thing. Use small random numbers.

✅ Recap

- The **loss** is one number saying how wrong the model is. MSE = mean of squared errors.

- The **slope (gradient)** says which way the loss goes up; we step the other way.
- **Gradient descent**: `w = w − lr × slope`, repeated many times.
- The **learning rate** must be not too small (slow) and not too big (diverges).
- A **2-layer network**: `H = sigmoid(X @ W1 + b1)`, `out = sigmoid(H @ W2 + b2)`.
- **Backpropagation** passes the blame from the output back through the layers to get every weight's slope.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Write the MSE loss

Finish the function `mse(y_true, y_pred)` so it returns the mean squared error: the average of `(y_pred − y_true)²`. It gets two NumPy arrays.

```
import numpy as np

def mse(y_true, y_pred):
    # Replace the line below
    return 0

print(mse(np.array([1.0, 2.0]), np.array([2.0, 4.0])))   # should be 2.5
```

Hints:

1. `y_pred - y_true` gives all the errors at once.
2. Square with `** 2`, then average with `np.mean(...)`.
3. `return np.mean((y_pred - y_true) ** 2)`

2. One step downhill

Using the café data from the lesson and a starting guess `w = 1.0`, do **one** gradient-descent step by hand:

- store the slope `mean(2 × error × cups)` in `slope` (the error is `w * cups - paid`)
- store the new weight `w − lr × slope` in `w_new`

```
import numpy as np

cups = np.array([1, 2, 3, 4, 5])
paid = np.array([2.9, 6.1, 9.0, 11.8, 15.2])
w = 1.0
lr = 0.01

# Calculate slope and w_new
```

Hints:

1. First `error = w * cups - paid` (an array of 5 errors).
2. `slope = np.mean(2 * error * cups)`
3. `w_new = w - lr * slope`. Since the slope is negative, `w_new` is bigger than 1.

3. Compare two learning rates

Finish the function `descend(lr, steps)`: start at `w = 0.0`, do `steps` gradient-descent steps with the given learning rate, and return the final `w`. Then store the loss after 20 steps with `lr = 0.002` in `loss_small` and with `lr = 0.03` in `loss_good`.

```
import numpy as np

cups = np.array([1, 2, 3, 4, 5])
paid = np.array([2.9, 6.1, 9.0, 11.8, 15.2])

def mse(y_true, y_pred):
    return np.mean((y_pred - y_true) ** 2)

def slope(w):
    return np.mean(2 * (w * cups - paid) * cups)

def descend(lr, steps):
    w = 0.0
    # Repeat the gradient-descent step here
    return w

loss_small = mse(paid, descend(0.002, 20) * cups)
# loss_good = ...
```

Hints:

1. Inside the function, loop `for i in range(steps):` and update `w` each time.
2. The update is `w = w - lr * slope(w)`.
3. `loss_good = mse(paid, descend(0.03, 20) * cups)`

4. Fit a line with gradient descent

Fit `y = a·x + b` to the made-up points below with gradient descent. The loop is there, but the two update lines are missing. Fill them in so `a` and `b` end up close to the best line. (Don't use `np.polyfit` here.)

```
import numpy as np

x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([1.2, 2.8, 5.1, 7.0, 8.9, 11.1])   # made-up points

a, b = 0.0, 0.0
lr = 0.05

for step_number in range(1000):
    error = a * x + b - y
    # Update a and b here

print(f"a = {a:.3f}, b = {b:.3f}")
```

Hints:

1. The slope for `a` is `np.mean(2 * error * x)`; the slope for `b` is `np.mean(2 * error)`.
2. Each one is updated with "minus learning rate times its slope", inside the loop.
3. `a = a - lr * np.mean(2 * error * x)` and `b = b - lr * np.mean(2 * error)`

5. Challenge: train the XOR network

Here is the 2-layer network with its forward pass and backpropagation already written. Only the **four update lines** are missing (for `W2`, `b2`, `W1` and `b1`). Add them so that after training the rounded outputs are `[0, 1, 1, 0]`.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

rng = np.random.default_rng(42)
W1 = rng.normal(size=(2, 4))
b1 = np.zeros(4)
W2 = rng.normal(size=(4, 1))
b2 = np.zeros(1)
lr = 1.0

for epoch in range(5000):
    H = sigmoid(X @ W1 + b1)
    out = sigmoid(H @ W2 + b2)

    d_out = (out - y) * out * (1 - out)
    d_hidden = (d_out @ W2.T) * H * (1 - H)

    grad_W2 = H.T @ d_out
    grad_b2 = d_out.sum(axis=0)
    grad_W1 = X.T @ d_hidden
    grad_b1 = d_hidden.sum(axis=0)

    # Update W2, b2, W1 and b1 here

print(np.round(out.ravel(), 3))
```

Hints:

1. Every weight array moves a small step against its own gradient: "minus learning rate times gradient".
2. The lines go inside the loop (indented with 4 spaces), e.g. `W2 -= lr * grad_W2`.
3. Do the same for `b2`, `W1` and `b1` with `grad_b2`, `grad_W1` and `grad_b1`.

➡️ Tomorrow

Recognising handwritten digits: XOR has only 4 examples. Tomorrow you'll point a neural network at 1797 small pictures of handwritten digits, learn how a picture becomes numbers, and train scikit-learn's `MLPClassifier` (the same kind of network you built today, ready-made) to read them.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Write the MSE loss

```
import numpy as np

def mse(y_true, y_pred):
    return np.mean((y_pred - y_true) ** 2)

print(mse(np.array([1.0, 2.0]), np.array([2.0, 4.0])))   # should be 2.5
```

2. One step downhill

```
import numpy as np

cups = np.array([1, 2, 3, 4, 5])
paid = np.array([2.9, 6.1, 9.0, 11.8, 15.2])
w = 1.0
lr = 0.01

error = w * cups - paid
slope = np.mean(2 * error * cups)
w_new = w - lr * slope
print(slope, w_new)
```

3. Compare two learning rates

```
import numpy as np

cups = np.array([1, 2, 3, 4, 5])
paid = np.array([2.9, 6.1, 9.0, 11.8, 15.2])

def mse(y_true, y_pred):
    return np.mean((y_pred - y_true) ** 2)

def slope(w):
    return np.mean(2 * (w * cups - paid) * cups)

def descend(lr, steps):
    w = 0.0
    for i in range(steps):
        w = w - lr * slope(w)
    return w

loss_small = mse(paid, descend(0.002, 20) * cups)
loss_good = mse(paid, descend(0.03, 20) * cups)
print(f"small lr: {loss_small:.3f}   good lr: {loss_good:.3f}")
```

4. Fit a line with gradient descent

```
import numpy as np

x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([1.2, 2.8, 5.1, 7.0, 8.9, 11.1])   # made-up points

a, b = 0.0, 0.0
lr = 0.05

for step_number in range(1000):
    error = a * x + b - y
    a = a - lr * np.mean(2 * error * x)
    b = b - lr * np.mean(2 * error)

print(f"a = {a:.3f}, b = {b:.3f}")
```

5. Challenge: train the XOR network

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

rng = np.random.default_rng(42)
W1 = rng.normal(size=(2, 4))
b1 = np.zeros(4)
W2 = rng.normal(size=(4, 1))
b2 = np.zeros(1)
lr = 1.0

for epoch in range(5000):
    H = sigmoid(X @ W1 + b1)
    out = sigmoid(H @ W2 + b2)

    d_out = (out - y) * out * (1 - out)
    d_hidden = (d_out @ W2.T) * H * (1 - H)

    grad_W2 = H.T @ d_out
    grad_b2 = d_out.sum(axis=0)
    grad_W1 = X.T @ d_hidden
    grad_b1 = d_hidden.sum(axis=0)

    W2 -= lr * grad_W2
    b2 -= lr * grad_b2
    W1 -= lr * grad_W1
    b1 -= lr * grad_b1

print(np.round(out.ravel(), 3))
```